Python Implementation in Botany

What is Python Implementation in Botany?

Python implementation in botany means using the Python programming language to **study plants scientifically through data analysis, plant growth modeling, image analysis, environmental monitoring, and AI-based plant research**.

Botanists collect large data about plant growth, climate conditions, soil quality, and plant diseases.
Python helps researchers **analyze plant data, predict growth patterns, detect diseases, and visualize plant development** easily.

It is widely used in:

- Plant science research
- Agriculture technology
- Environmental studies
- Plant disease detection
- Botanical data analysis
- AI in agriculture

Why Python is Used in Botany

Python helps botanists to:

- Analyze plant growth data
- Study environmental effects on plants
- Identify plant diseases using AI
- Monitor soil & weather conditions
- Visualize plant development graphs
- Automate botanical research tasks

Popular libraries:

- **Pandas / NumPy** – plant data analysis
- **Matplotlib** – plant growth graphs
- **OpenCV** – plant image analysis
- **Scikit-learn** – AI plant disease prediction

Program 1 — Plant Growth Calculator (Used for: Growth experiments)

```
initial_height = float(input("Enter initial plant height (cm): "))
final_height = float(input("Enter final plant height (cm): "))
days = int(input("Enter number of days: "))

growth_rate = (final_height - initial_height) / days

print("Plant Growth Rate per Day:", growth_rate,"cm")
```

Program 2 — Leaf Area Calculator (Used in plant physiology studies)

```
length = float(input("Enter leaf length (cm): "))
width = float(input("Enter leaf width (cm): "))

leaf_area = length * width * 0.75

print("Estimated Leaf Area:", leaf_area,"sq.cm")
```

Program 3 — Plant Population Density (Used in ecology research)

```
plants = int(input("Number of plants: "))
area = float(input("Area in square meters: "))

density = plants/area

print("Plant Population Density:",density,"plants per sq.m")
```

Program 4 — Plant DNA GC Content (Used in plant genetics (Used in plant genetics)

```
dna = input("Enter Plant DNA Sequence: ").upper()

g = dna.count('G')
c = dna.count('C')

gc_content = (g+c)/len(dna)*100

print("GC Content:",gc_content,"%")
```

Program 5 — Logistic Plant Growth Model (Used in ecological modelling)

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0,10,100)

K = 100
r = 0.5
P0 = 10

P = K/(1+((K-P0)/P0)*np.exp(-r*t))

plt.plot(t,P)
plt.xlabel("Time")
plt.ylabel("Plant Population")
plt.title("Plant Growth Model")
plt.show()
```

Program 6 — Photosynthesis Rate Calculator (Used in Plant physiology experiments)

```
light = float(input("Enter light intensity: "))
co2 = float(input("Enter CO2 level: "))

rate = (light * co2) / 100

print("Photosynthesis Rate:", rate)
```

Program 7 — Seed Germination Percentage (Used in: Agricultural botany)

```
total_seeds = int(input("Total Seeds: "))
germinated = int(input("Germinated Seeds: "))

percentage = (germinated/total_seeds)*100

print("Germination Percentage:",percentage,"%")
```

Program 8 — Chlorophyll Content Estimator (Used in: Plant biochemistry)

```
absorbance = float(input("Enter absorbance value: "))

chlorophyll = 20 * absorbance

print("Estimated Chlorophyll Content:",chlorophyll)
```

Program 9 — Plant Species Similarity (DNA Sequence Comparison Used in phylogenetics & evolutionary botany)

```
seq1 = input("Enter Plant DNA Sequence 1: ").upper()
seq2 = input("Enter Plant DNA Sequence 2: ").upper()

matches = 0

for a,b in zip(seq1,seq2):
    if a == b:
        matches += 1

similarity = matches/len(seq1)*100

print("Genetic Similarity:",similarity,"%")
```

Program 10 — NDVI Vegetation Index Calculator (Used in remote sensing & ecological monitoring)

```
nir = float(input("Enter Near Infrared Value: "))
red = float(input("Enter Red Light Value: "))

ndvi = (nir-red)/(nir+red)

print("NDVI Vegetation Index:",ndvi)
```

Program 11 — Plant Growth Prediction using Linear Regression(Used in Growth modelling,Predictive botany)

```python
import numpy as np
from sklearn.linear_model import LinearRegression

days = np.array([1,2,3,4,5]).reshape(-1,1)
height = np.array([5,7,9,11,13])

model = LinearRegression()
model.fit(days,height)

predicted = model.predict([[6]])

print("Predicted Plant Height on Day 6:",predicted[0])
```

Program 12 — Plant Biodiversity Index (Shannon Index Used in: Ecology research, Forest biodiversity analysis)

```python
import math

species = [10,20,30,40]

total = sum(species)
H = 0

for count in species:
    p = count/total
    H -= p*math.log(p)

print("Shannon Biodiversity Index:",H)
```

Program 13 — Plant Disease Detection (Simple ML Model Used in: Smart agriculture,Plant pathology)

```python
from sklearn.tree import DecisionTreeClassifier

humidity = [[80],[60],[90],[50]]
disease = [1,0,1,0]

model = DecisionTreeClassifier()
model.fit(humidity,disease)

prediction = model.predict([[85]])

print("Disease Prediction (1=Disease,0=No Disease):",prediction[0])
```

Mini Project

Project Title:

AI-Based Plant Disease Detection using Python

```python
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# ---------------- DATASET ----------------
data = {
    "Humidity":[80,60,90,50,85,70,95,40],
    "Temperature":[30,25,32,20,31,26,35,18],
    "Leaf_Spots":[1,0,1,0,1,0,1,0],
    "Disease":[1,0,1,0,1,0,1,0]
}

df = pd.DataFrame(data)

X = df[["Humidity","Temperature","Leaf_Spots"]]
y = df["Disease"]
```

```python
# ---------------- TRAIN TEST SPLIT ----------------
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2)

# ---------------- MODEL ----------------
model = DecisionTreeClassifier()
model.fit(X_train,y_train)

# ---------------- PREDICTION ----------------
prediction = model.predict([[85,31,1]])

print("Disease Prediction (1=Disease,0=Healthy):",prediction[0])

# ---------------- ACCURACY ----------------
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test,y_pred)

print("Model Accuracy:",accuracy)
```